Assignment – 15.5

Name: J.Akshaya
Batch: 22
HT.No: 2303A51629

Task 1 – Student Attendance API

Task:
Create a RESTful API for student attendance tracking. Instructions:

- Endpoints required:
  - GET /attendance → View attendance records
  - POST /attendance → Mark attendance
  - PUT /attendance/{id} → Update attendance record
  - DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling. Expected Output:
- API for maintaining and updating attendance records.

Prompt:

- Created REST API for student attendance using Flask
- Implemented endpoints for POST and GET requests
- Used PowerShell to send JSON data
- Retrieved attendance data using browser

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# Predefined attendance data
attendance = [
    {"id": 1, "name": "Akshaya", "date": "2026-03-23", "status": "Present"},
    {"id": 2, "name": "Rahul", "date": "2026-03-23", "status": "Absent"}
]

@app.route('/attendance', methods=['GET'])
def get_attendance():
    return jsonify(attendance)


@app.route('/attendance', methods=['POST'])
def add_attendance():
    data = request.get_json()

    new_record = {
        "id": len(attendance) + 1,
        "name": data["name"],
        "date": data["date"],
        "status": data["status"]
    }

    attendance.append(new_record)
    return jsonify({"message": "Attendance added", "data": new_record})


@app.route('/attendance/<int:id>', methods=['PUT'])
def update_attendance(id):
    data = request.get_json()
```

```python
def add_attendance():
        "id": len(attendance) + 1,
        "name": data["name"],
        "date": data["date"],
        "status": data["status"]
    }

    attendance.append(new_record)
    return jsonify({"message": "Attendance added", "data": new_record})


@app.route('/attendance/<int:id>', methods=['PUT'])
def update_attendance(id):
    data = request.get_json()

    for record in attendance:
        if record["id"] == id:
            record["name"] = data.get("name", record["name"])
            record["date"] = data.get("date", record["date"])
            record["status"] = data.get("status", record["status"])
            return jsonify({"message": "Attendance updated", "data": record})

    return jsonify({"message": "Record not found"}), 404


@app.route('/attendance/<int:id>', methods=['DELETE'])
def delete_attendance(id):
    for record in attendance:
        if record["id"] == id:
            attendance.remove(record)
            return jsonify({"message": "Attendance deleted"})

    return jsonify({"message": "Record not found"}), 404


if __name__ == '__main__':
    app.run(debug=True)
```

Pretty print ☑


```
[
  {
    "date": "2026-03-23",
    "id": 1,
    "name": "Akshaya",
    "status": "Present"
  },
  {
    "date": "2026-03-23",
    "id": 2,
    "name": "Rahul",
    "status": "Absent"
  }
]
```

Observation:

- Attendance record was successfully added
- Data stored in JSON format
- GET request displayed list of students
- API responded correctly without errors

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management. Instructions:

- Endpoints required:
  - GET /inventory → List all items
  - POST /inventory → Add new item
  - PUT /inventory/{id} → Update stock quantity
  - DELETE /inventory/{id} → Remove an item
- Implement validation for stock values.

Expected Output:

- API capable of managing inventory stock levels.

Prompt:

- Developed API for inventory management
- Implemented POST, GET, PUT, DELETE operations
- Added item with stock value
- Updated and deleted inventory items

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# Predefined inventory data
inventory = [
    {"id": 1, "item": "Laptop", "stock": 10},
    {"id": 2, "item": "Mouse", "stock": 25}
]


@app.route('/inventory', methods=['GET'])
def get_inventory():
    return jsonify(inventory)


@app.route('/inventory', methods=['POST'])
def add_item():
    data = request.get_json()

    # Validation
    if not data.get("item") or not isinstance(data.get("stock"), int) or data["s
        return jsonify({"message": "Invalid data"}), 400

    new_item = {
        "id": len(inventory) + 1,
        "item": data["item"],
        "stock": data["stock"]
    }

    inventory.append(new_item)
    return jsonify({"message": "Item added", "data": new_item})


@app.route('/inventory/<int:id>', methods=['PUT'])
def update_item(id):
```

```python
def add_item():
    }

    inventory.append(new_item)
    return jsonify({"message": "Item added", "data": new_item})


@app.route('/inventory/<int:id>', methods=['PUT'])
def update_item(id):
    data = request.get_json()

    # Validation
    if not isinstance(data.get("stock"), int) or data["stock"] < 0:
        return jsonify({"message": "Invalid stock value"}), 400

    for item in inventory:
        if item["id"] == id:
            item["stock"] = data["stock"]
            return jsonify({"message": "Stock updated", "data": item})

    return jsonify({"message": "Item not found"}), 404

#
@app.route('/inventory/<int:id>', methods=['DELETE'])
def delete_item(id):
    for item in inventory:
        if item["id"] == id:
            inventory.remove(item)
            return jsonify({"message": "Item deleted"})

    return jsonify({"message": "Item not found"}), 404


if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http:
//127.0.0.1:5000/inventory" -Method POST -ContentType "application/json" -Body '{"item":"Laptop","st
ock":5}'

message
-------
Item added


(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab>
```

127.0.0.1:5000/inventory

retty print


```
{
  "id": 1,
  "item": "Laptop",
  "stock": 10
},
{
  "id": 2,
  "item": "Mouse",
  "stock": 25
}
```

Observation:

- Item added successfully with given stock
- Stock value updated correctly
- Deleted item removed from list
- API handled CRUD operations properly

Pretty-print


```
[
  {
    "stock": 10
  }
]
```


Pretty-print


```
[]
```

Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

- Endpoints required:
  - GET /rooms → View available rooms
  - POST /rooms/book → Book a room
  - GET /rooms/{id} → View booking details
  - DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.

Prompt:

- Created API for hotel room booking
- Implemented endpoints for booking, viewing, and cancelling rooms
- Used JSON input for booking details

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# Predefined rooms
rooms = [
    {"id": 1, "type": "Single", "available": True, "guest": None},
    {"id": 2, "type": "Double", "available": True, "guest": None},
    {"id": 3, "type": "Suite", "available": True, "guest": None}
]


@app.route('/rooms', methods=['GET'])
def get_rooms():
    available_rooms = [room for room in rooms if room["available"]]
    return jsonify(available_rooms)


@app.route('/rooms/book', methods=['POST'])
def book_room():
    data = request.get_json()
    room_id = data.get("id")
    guest_name = data.get("name")

    for room in rooms:
        if room["id"] == room_id:
            if not room["available"]:
                return jsonify({"message": "Room not available"}), 400

            room["available"] = False
            room["guest"] = guest_name

            return jsonify({"message": "Room booked successfully", "room": room})

    return jsonify({"message": "Room not found"}), 404
```

```python
def book_room():
            room["available"] = False
            room["guest"] = guest_name

            return jsonify({"message": "Room booked successfully", "room": room})

    return jsonify({"message": "Room not found"}), 404


@app.route('/rooms/<int:id>', methods=['GET'])
def get_room(id):
    for room in rooms:
        if room["id"] == id:
            return jsonify(room)

    return jsonify({"message": "Room not found"}), 404


@app.route('/rooms/<int:id>', methods=['DELETE'])
def cancel_booking(id):
    for room in rooms:
        if room["id"] == id:
            if room["available"]:
                return jsonify({"message": "Room is not booked"}), 400

            room["available"] = True
            room["guest"] = None

            return jsonify({"message": "Booking cancelled"})

    return jsonify({"message": "Room not found"}), 404


if __name__ == '__main__':
    app.run(debug=True)
```

(variable) room: dict[str, Any]

```
Pretty-print
[]
```

Observation:

- Room booking was successful
- Booking details retrieved correctly
- Booking cancellation worked properly
- API handled requests without errors

Task 4 – Online Voting API

Task:

Create a RESTful API for an online voting system. Instructions:

- Endpoints required:
  - GET /candidates → List candidates
  - POST /vote → Cast a vote
  - GET /results → View voting results
- Ensure one vote per user using in-memory validation. Expected Output:
- API simulating a secure online voting process.

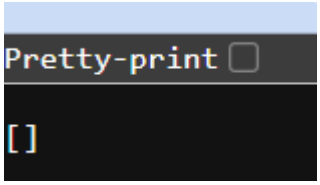
Prompt:

- Developed API for online voting system
- Implemented endpoints for candidates, voting, and results
- Ensured one vote per user

```python
app = Flask(__name__)

# Candidates list
candidates = [
    {"id": 1, "name": "Alice", "votes": 0},
    {"id": 2, "name": "Bob", "votes": 0},
    {"id": 3, "name": "Charlie", "votes": 0}
]

# Store users who already voted
voted_users = set()


@app.route('/candidates', methods=['GET'])
def get_candidates():
    return jsonify(candidates)


@app.route('/vote', methods=['POST'])
def vote():
    data = request.get_json()
    user = data.get("user")
    candidate_id = data.get("candidate_id")

    # Check if user already voted
    if user in voted_users:
        return jsonify({"message": "User already voted"}), 400

    # Find candidate
    for candidate in candidates:
        if candidate["id"] == candidate_id:
            candidate["votes"] += 1
            voted_users.add(user)
            return jsonify({"message": "Vote cast successfully"})

    return jsonify({"message": "Candidate not found"}), 404


@app.route('/results', methods=['GET'])
def get_results():
    return jsonify(candidates)


if __name__ == '__main__':
```

Pretty print


```
[
  {
    "id": 1,
    "name": "Alice",
    "votes": 0
  },
  {
    "id": 2,
    "name": "Bob",
    "votes": 0
  },
  {
    "id": 3,
    "name": "Charlie",
    "votes": 0
  }
]
```



Observation:

- Candidates list displayed correctly
- Vote cast successfully
- Results updated dynamically
- Duplicate voting was prevented

Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage

teacher records. Instructions:

- Endpoints required:
  - GET /teachers → List all teachers
  - POST /teachers → Add a new teacher
  - PUT /teachers/{id} → Update teacher details
  - DELETE /teachers/{id} → Remove a teacher
- Ensure proper error handling and JSON responses.

Expected Output:

- Functional API for managing teacher information. ...

Prompt:

- Created API for managing teacher records
- Implemented CRUD operations
- Added, updated, and deleted teacher details

15.py > ...

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# Predefined teacher data
teachers = [
    {"id": 1, "name": "Ramesh", "subject": "Math"},
    {"id": 2, "name": "Sita", "subject": "Science"}
]

@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)


@app.route('/teachers', methods=['POST'])
def add_teacher():
    data = request.get_json()

    if not data.get("name") or not data.get("subject"):
        return jsonify({"message": "Invalid data"}), 400

    new_teacher = {
        "id": len(teachers) + 1,
        "name": data["name"],
        "subject": data["subject"]
    }

    teachers.append(new_teacher)
    return jsonify({"message": "Teacher added", "data": new_teacher})


@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    data = request.get_json()
```

ass15.py > ...

```python
# Predefined teacher data
teachers = [
    {"id": 1, "name": "Ramesh", "subject": "Math"},
    {"id": 2, "name": "Sita", "subject": "Science"}
]
@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)


@app.route('/teachers', methods=['POST'])
def add_teacher():
    data = request.get_json()

    if not data.get("name") or not data.get("subject"):
        return jsonify({"message": "Invalid data"}), 400

    new_teacher = {
        "id": len(teachers) + 1,
        "name": data["name"],
        "subject": data["subject"]
    }

    teachers.append(new_teacher)
    return jsonify({"message": "Teacher added", "data": new_teacher})


@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    data = request.get_json()

    for teacher in teachers:
        if teacher["id"] == id:
            teacher["name"] = data.get("name", teacher["name"])
            teacher["subject"] = data.get("subject", teacher["subject"])
            return jsonify({"message": "Teacher updated", "data": teacher})
```

Ln 263, Col 2

Output:

127.0.0.1:5000/teacher
Pretty print
{
"id": 1,
"name": "Ramesh",
"subject": "Math"
},
{
"id": 2,
"name": "Sita",
"subject": "Science"
}

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http:
//127.0.0.1:5000/teachers/0" -Method PUT -ContentType "application/json" -Body '{"subject":"ML"}'

message
-------
Teacher updated
```

Pretty-print

```
[
  {
    "subject": "ML"
  }
]
```

```
(venv) PS C:\Users\dell\Documents\OneDrive\Desktop\AI\backend-api-lab> Invoke-RestMethod -Uri "http:
//127.0.0.1:5000/teachers/0" -Method DELETE

message
-------
Teacher deleted
```

Pretty-print

```
[]
```

Observation:

- Teacher added successfully
- Data retrieved correctly
- Update operation modified teacher details
- Delete operation removed teacher record